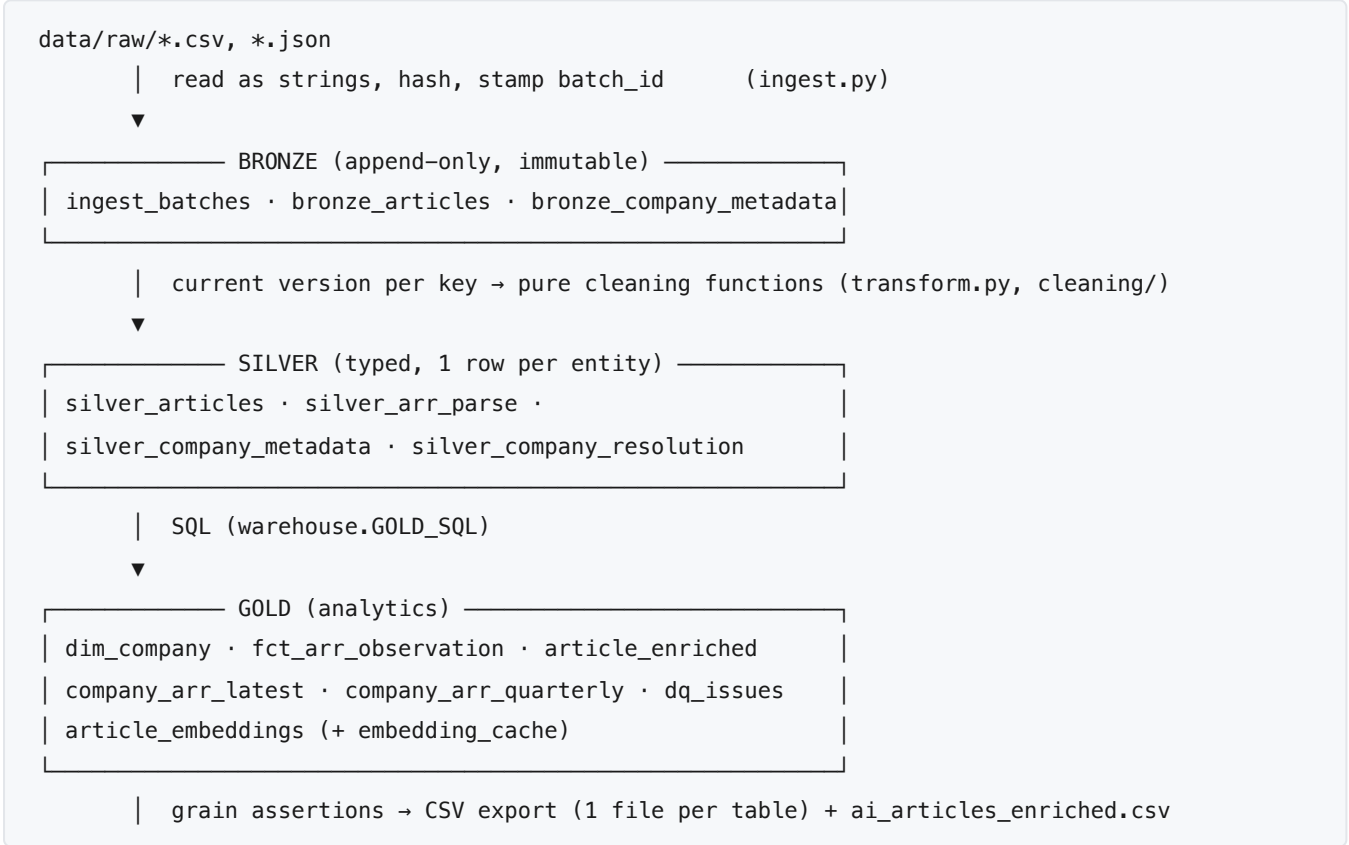
Data Architecture: Tech-News ARR Observations

1. What the pipeline produces

The pipeline turns 750 messy tech-news articles plus a company metadata file into:

1. **A queryable model of company ARR observations over time.** Each valid `revenue` string becomes one dated, USD-normalized ARR observation that links back to its source article.
2. **A semantic search index.** Article embeddings live next to the relational data so SQL filters and vector similarity can run in one query.

Core principle: an article ARR value is a *claim*, not company master data. No table says "Company X's ARR is Y" without carrying the article, the raw string, the FX rate and the parser version that produced it. "Latest" and "quarterly" ARR are derived views with explicit tie-break rules. They are not overwritten attributes on the company.



Store: a single DuckDB file (`output/warehouse.duckdb`). It is an embedded columnar engine with window functions, `INSERT OR IGNORE`, and a native `FLOAT[384]` array type with `array_cosine_similarity`, which makes hybrid search one SQL statement.

2. Table catalog

Every table has a declared grain in `warehouse.TABLE_GRAIN`. After every run, before any export, the pipeline checks that grain: no duplicate keys and no null keys. A violation fails the run.

Layer	Table	Grain (unique key)	Purpose
meta	`ingest_batches`	`batch_id`	One row per ingested file. Holds `file_sha256`, `batch_seq` (load order), row count, new versions and ingest time.
bronze	`bronze_articles`	`(article_id, record_hash)`	Every distinct **version** of every source row, stored as raw strings. Unknown columns go to `extra_columns_json`.
bronze	`bronze_company_metadata`	`(company_name, record_hash)`	Every distinct version of each metadata entry, plus `raw_json`.
silver	`silver_articles`	`article_id`	Current version of each article, typed. Raw and clean values sit side by side (`published_date_raw` / `published_date` / `date_status` / `date_format` / `date_is_ambiguous` / `date_alternative`, `category_raw` / `category`, `company_name_raw` / `company_id` / `company_match_method`).
silver	`silver_arr_parse`	`article_id`	Revenue parse result for **every** article, including failures: `parse_status`, `arr_usd`, low/high bounds, `is_range`, `currency_original`, `amount_original`, `fx_rate_to_usd`, `fx_rate_set`, `error_reason`, `parser_version`. This is where you debug a failed parse.
silver	`silver_company_resolution`	`company_name_raw`	The entity-resolution decision for each distinct raw company string: method (`exact`, `normalized`, `alias`, `fuzzy`, `ambiguous`, `unmatched`), score, `in_metadata`, `relationship`, `article_count`.
silver	`silver_company_metadata`	`company_id`	Typed metadata, `industry` mapped to the shared taxonomy, `company_size_category`, `metadata_quality_notes`.
gold	`dim_company`	`company_id`	One row per resolved company identity: the 21 metadata companies plus 5 article-only companies (`in_metadata = false`).
gold	**`fct_arr_observation`**	`observation_id` (= `'ARR-' ‖ article_id`)	One row per valid reported ARR observation, i.e. one article reporting ARR for one company as of its `published_date`.
gold	`article_enriched`	`article_id`	Wide, filter-friendly table: article + ARR status/value + company metadata + `company_age` + `is_ai_related`.
gold	`company_arr_latest`	`company_id`	Derived "latest reported ARR" per company, with the observation it came from.
gold	`company_arr_quarterly`	`(company_id, year_quarter)`	End-of-quarter ARR (last observation in the quarter), plus min, max, median and count.
gold	`dq_issues`	`(entity_type, entity_id, issue_type)`	One row per excluded or suspicious record and the reason.

Layer	Table	Grain (unique key)	Purpose
gold	`article_embeddings`	`article_id`	`embedding FLOAT[384]`, `model_name`, `text_hash`, `top_similar_articles`, `top_similar_scores`.
cache	`embedding_cache`	`(text_hash, model_name)`	Content-addressed embedding store, so text is only embedded once. Not exported.
meta	`pipeline_runs`	`run_id`	Run log: timings, parser version, model, batches, row counts.

Relationships

```
dim_company 1 ──< fct_arr_observation >── 1 silver_articles ── 1 article_enriched
     │                    │                       │
     │                    │                       └── (article_id, source_record_hash) ──>
bronze_articles
     │                    └── source_batch_id ──> ingest_batches
     └──< silver_company_resolution (raw name → company_id)
article_embeddings 1 ── 1 article_enriched
```

Lineage: from a number back to the source row

```
SELECT f.observation_id, f.company_id, f.as_of_date, f.arr_usd,
       f.revenue_raw, f.currency_original, f.fx_rate_to_usd, f.parser_version,
       b.source_file, b.source_row_number, b.revenue AS revenue_exactly_as_received,
b.published_date
FROM fct_arr_observation f
JOIN bronze_articles b ON b.article_id = f.article_id AND b.record_hash = f.source_record_hash
WHERE f.observation_id = 'ARR-ART0004';
```

The join on `record_hash` points to the **exact version** of the row that produced the number, even after later batches changed the article.

3. ARR semantics

What counts as an observation

A row enters `fct_arr_observation` only if all three of these hold:

Condition	If it fails
`silver_arr_parse.parse_status = 'valid'`	`dq_issues.issue_type ∈ {arr_missing, arr_not_available, arr_not_disclosed, arr_unparseable}`
`silver_articles.date_status = 'valid'`	`date_invalid` / `date_out_of_range` / `date_missing` and `arr_valid_but_not_modeled`
`company_id IS NOT NULL`	`company_ambiguous` / `company_missing` and `arr_valid_but_not_modeled`

Missing, `N/A` and `Not disclosed` values are **classified, never coerced to 0 or dropped**. They stay visible in `silver_arr_parse` and `article_enriched.arr_status`, and are explained in `dq_issues`.

Results on the provided data:

Status	Count
valid	558 (400 USD, 55 JPY, 53 EUR, 50 GBP; 58 ranges)
not_disclosed	85
missing	66
not_available	41

All 558 valid values become observations.

Deriving "latest" and "quarterly" ARR

- **`company_arr_latest`**: the observation with the maximum `as_of_date`. Same-day ties prefer an unambiguous date, then the highest `article_id`. The table also exposes `previous_arr_usd`, `first_as_of_date` and `observation_count`.
- **`company_arr_quarterly`**: only quarters **that have an observation**. There is no forward fill. `arr_usd_end_of_quarter` is the last observation in the quarter (same tie-break as latest). The table also exposes `prior_observed_quarter_arr_usd` for QoQ comparisons. To get a dense quarterly series, forward-fill explicitly at query time (§7). Silently carrying a stale value forward would hide gaps.
- Both views use only `company_relationship = 'self'` rows. A name configured as a product or segment of a parent (`product_of_parent`) stays in the fact table but cannot pose as the parent's total ARR.

Caveats that travel with the data

- `date_is_ambiguous` and `date_alternative` sit on every observation. Both readings of an ambiguous date are always in the same year, so year filters are safe, but quarter and month may not be.
- `is_range` with `arr_usd_low` / `arr_usd_high`: the midpoint is a modeling choice, and the bounds are kept.
- `fx_rate_set = 'assignment_static_v1'`: fixed rates (EUR×1.1, GBP×1.27, JPY÷150), not historical spot rates.
- `same_day_observation_count > 1`: several articles reported on the same day (12 such rows). In this data they agree.

4. Cleaning rules and assumptions

Revenue → `arr_usd` (`cleaning/revenue.py`)

- Grammar: `[currency] number [scale] [currency]`, optionally as a range `A – B`, `A — B` or `A to B`.
- Currency can be `$ € £ ¥`, `US$`, or the codes USD/EUR/GBP/JPY.
- Scale can be `k/M/B/T`, `mm/mn/bn/tn`, or `thousand/million/billion/trillion`.
- A side of a range without its own currency or scale inherits it from the other side (`$10 – 20M`).
- Arithmetic is done in `Decimal`, and the result is rounded half-up to a Python `int` **once, at the end**. This avoids float drift on JPY ÷ 150.

- Thousands separators must be well-formed (`1,234,567`). `2,1B` is read as a European decimal comma. `1,23,45` is rejected.
- Negative, zero, mixed-currency and malformed values become `unparseable` with an `error_reason`.
- No currency means USD.

`published_date` (`cleaning/dates.py`)

Formats are matched explicitly; nothing is guessed. Each row records which format matched. Observed formats: ISO date (109), ISO datetime with `Z` (123, reduced to its UTC date), `D Mon YYYY` (141), `Month D, YYYY` (120), `MM/DD/YYYY` (111), `DD-MM-YYYY` (129) and `MM-DD-YYYY` (17).

Ambiguous numeric dates (both leading fields ≤ 12; 93 rows):

Separator	Rule	Evidence in the source
`/`	US `MM/DD`	No slash date has first field > 12, and 71 have second field > 12.
`-`	EU `DD-MM`	63 unambiguous dash dates are day-first and 17 are month-first, so dash dates are mixed and DD-MM is only the majority.

The ARR series is almost perfectly monotonic: 7 decreases in 460 consecutive unambiguous observations. That allows an independent check of each ambiguous row: does the chosen reading or the alternative fit the company's growth curve?

Separator	Fits chosen reading only	Fits alternative only	Fits both	Fits neither
`/`	16	0	14	0
`-`	7	5	28	2

The slash rule is well supported. The dash rule is only marginally better than the alternative. That is why every ambiguous row is flagged and keeps `date_alternative`, rather than silently trusting a convention. A possible next step is to use the growth-consistency signal to resolve dash dates. It is not applied automatically because it uses the metric itself to fix its own timestamp, which is circular, so it would need human review.

Invalid calendar dates (`2023-02-30`) become `invalid`. Years before 1990 or dates in the future become `out_of_range`. Empty values become `missing`.

Category taxonomy (`cleaning/categories.py`)

One taxonomy covers both article `category` and metadata `industry`, so the AI filter is simply `category = 'AI_ML' OR industry = 'AI_ML'`. The mapping (`CATEGORY_MAPPING`):

Taxonomy value	Raw labels
`AI_ML`	AI/ML, AI & ML, Artificial Intelligence, Machine Learning
`CLOUD`	Cloud, Cloud Computing, Cloud Services
`DATA_ANALYTICS`	Analytics, Data Analytics, Big Data
`FINTECH`	FinTech, Financial Technology, Finance
`CYBERSECURITY`	Cybersecurity, InfoSec, Security
`SOFTWARE_SAAS`	Software, Enterprise Software, SaaS

Unknown labels become `OTHER`, keep their raw value, and are flagged.

Company resolution (`cleaning/companies.py`)

The resolver applies a cascade and records which step matched:

1. `exact`
2. `alias`: the curated table, checked first
3. `normalized`: case and legal suffixes removed (Inc., Corporation, Technologies), parentheticals and spaces stripped. Examples: "Open AI", "Mongo DB", "NVIDIA Corporation", "Amazon Web Services (AWS)".
4. `ambiguous`: multi-entity names such as `A / B` or `A & B` are **not attributed**.
5. `fuzzy`: difflib ratio ≥ 0.88 **and** at least 0.05 ahead of the runner-up. This prevents matches like "Cohere"→"Confluent".
6. `unmatched`: the name still gets a stable slug `company_id`, so its ARR series is queryable, with `in_metadata = false`.

Results: 46 raw names resolve to 26 companies.

- Unmatched: Cohere, xAI, Mistral AI, Hugging Face and Perplexity AI (25 articles). These are exported to `output/unmatched_companies.csv` and logged as a warning.
- **Reviewed alias decisions**, backed by data:
- `Azure` / `Microsoft Azure` → Microsoft. Their ARR values sit exactly on Microsoft's series ($79.17B, $83.85B), so they are name variants here, not segment revenue.
- `The Boring Company / SpaceX` → SpaceX. All three values are on SpaceX's series.

A new, unreviewed compound name still stays unresolved.

Metadata

The metadata is used as given, as the enrichment source of truth. Some values look synthetic, for example OpenAI with industry "Data Analytics" and Microsoft founded in 2018. Only internal contradictions are flagged: `ticker_on_private_company` (4) and `public_company_without_ticker` (5).

- `company_age` = `published_year − founded_year`. A negative result is set to null and flagged.
- Size thresholds: Small < 10,000 ≤ Medium ≤ 30,000 < Large.

5. Re-runs, incremental batches, backfills, schema change

Idempotency

- `batch_id = <file stem>-<sha256[:12]>`, so re-delivering the same file gives the same batch id.
- Bronze inserts are `INSERT OR IGNORE` on `(natural key, record_hash)`, so a repeated record is a no-op.
- Silver and gold are rebuilt with `CREATE OR REPLACE` **in one transaction** from the current bronze versions. All keys are deterministic: `observation_id = 'ARR-' ‖ article_id` and `company_id` is a slug. No sequences or UUIDs are used in modeled tables.
- Grain assertions run before export.
- Tested: running twice produces identical row counts and adds 0 new versions and 0 new embeddings. A full refresh produces a byte-identical `fct_arr_observation`.

New batches

```
python -m arr_pipeline run --articles data/raw/batch_2026_10_12.csv
```

- New articles are appended.
- A changed article (same `article_id`, different content) becomes a **new version** in bronze. Silver picks the version from the latest `batch_seq`, and the older version stays in bronze for audit.
- Only new text is embedded, thanks to the content-hash cache.

Backfills

Parsing rules live in versioned code (`PARSER_VERSION`, `FX_RATE_SET`, the taxonomy and the alias table). To backfill:

1. Change the rule and bump `PARSER_VERSION`.
2. Re-run.

Silver and gold are re-derived from the **immutable bronze**, so history is re-interpreted consistently. Every row carries the `parser_version` that produced it. `--full-refresh` rebuilds everything from the raw files.

At larger scale, the same idea applies with partitioning: re-derive only the partitions (by `batch_id` or publish month) whose inputs or rules changed.

Schema changes

- **New source column**: bronze keeps it in `extra_columns_json` and loses nothing. Promoting it to a typed column is an explicit change: `ALTER TABLE ... ADD COLUMN`, then a backfill.
- **Missing required column**: ingestion fails fast with a clear error before anything is written. This is a data contract, not a silent null.
- **Renamed column**: add it to `ingest.COLUMN_RENAMES` (the source-to-canonical column map) and backfill.
- **Embedding model change**: the cache is keyed by `(text_hash, model_name)`, so a new model coexists with the old one until the switch.

6. Semantic search

- **Text**: `title + ". " + summary`, embedded with `sentence-transformers/all-MiniLM-L6-v2` (384 dimensions). Vectors are L2-normalized, so cosine similarity equals the dot product.
- **Storage**:
- DuckDB `FLOAT[384]` columns (`article_embeddings`, `embedding_cache`)
- `output/embeddings/article_embeddings.npy` plus `article_ids.json`, a reusable numpy form
- JSON arrays in the CSV exports
- **`find_similar_articles(query_text, top_k=5)`** returns `[(article_id, cosine_score), ...]`. `ArticleIndex.find_similar_to_article(article_id)` does the same for an existing article and excludes the article itself.
- **`hybrid_search(con, query_text | similar_to_article_id, min_year, max_year, categories, industries, ai_related, min_arr_usd, company_ids, top_k)`** runs one parameterized SQL statement: filters on `article_enriched` plus `ORDER BY array_cosine_similarity(...)`.
- **`top_similar_articles`**: the 3 nearest *other* articles for every article. Scores are rounded to 6 decimal places before ranking, and ties break by `article_id`. Determinism matters here because the corpus has many identical templated summaries.

- **Caveat**: the summaries are drawn from about 12 templates, so many neighbors are "same template" matches. With real article bodies, retrieval quality would mostly come from the body text.
- **Offline fallback**: `--embedder hashing` is a deterministic bag-of-words embedder. Tests use it so the suite runs in about 3 seconds without downloading a model. It is lexical, not semantic.

7. Example queries

```
-- ARR over time for one company (the primary use case)
SELECT as_of_date, arr_usd, revenue_raw, date_is_ambiguous, article_id, source_url
FROM fct_arr_observation WHERE company_id = 'openai' ORDER BY as_of_date;

-- Dense quarterly series with explicit, visible forward fill
WITH cal AS (
  SELECT DISTINCT c.company_id, y.year, q.quarter
  FROM dim_company c, range(2020, 2025) y(year), range(1, 5) q(quarter)
)
SELECT cal.company_id, cal.year, cal.quarter, q.arr_usd_end_of_quarter AS observed,
       last_value(q.arr_usd_end_of_quarter IGNORE NULLS) OVER (
         PARTITION BY cal.company_id ORDER BY cal.year, cal.quarter) AS arr_usd_ffill
FROM cal LEFT JOIN company_arr_quarterly q USING (company_id, year, quarter)
WHERE cal.company_id = 'openai' ORDER BY 2, 3;

-- Filter articles by date, category, industry and ARR threshold
SELECT article_id, title, company_name, published_date, category, industry, arr_usd
FROM article_enriched
WHERE published_date BETWEEN DATE '2023-01-01' AND DATE '2023-12-31'
  AND (category = 'AI_ML' OR industry = 'AI_ML') AND arr_status = 'valid' AND arr_usd > 1e9;

-- Why is this article not in the fact table?
SELECT * FROM dq_issues WHERE entity_id = 'ART0001';
```

8. Running reliably beyond a local batch

Concern	Local implementation	Production design
Landing	Files in `data/raw/`	Object storage (S3/GCS) partitioned by `ingest_date`, with immutable objects and event notifications.
Orchestration	CLI run	Airflow or Dagster job per batch: `ingest → silver → gold → embed → checks → publish`. Each task is idempotent and retries are safe because every write is keyed.
Storage	One DuckDB file	Bronze, silver and gold as Iceberg or Delta tables (Snowflake/Databricks or DuckDB over Iceberg). `MERGE` on the same keys replaces full rebuilds, and the time-travel features are used for audit.
Incrementality	Full silver/gold rebuild (750 rows takes under 1 s)	Process only changed keys: the article_ids in new batches and the companies whose metadata or alias changed. Recompute latest and quarterly views only for affected `company_id`s.

Concern	Local implementation	Production design
Data contracts	Required-column check and grain assertions	The same checks as blocking tasks (dbt tests or Great Expectations), plus thresholds such as `unparseable_rate < 2%`, `unmatched_company_rate < 5%` and `ambiguous_date_rate`. Breaches page someone and **stop publish**, so consumers keep the last good gold snapshot.
Monitoring	`pipeline_runs`, `dq_issues`, log warnings	Run-level metrics such as rows per status, new versions, parse-failure rate and embedding latency, plus freshness SLAs per table. `dq_issues` trends go on a dashboard.
Entity resolution	Code alias table plus a fuzzy threshold	A reviewed alias table as data with owner, decided_at and evidence. A review queue for `unmatched`, `ambiguous` and `fuzzy` decisions. Point-in-time alias validity for M&A and renames.
FX	Static assignment rates	A `dim_fx_rate(currency, date, rate, source)` table joined on `as_of_date`, with `fx_rate_set` kept for reproducibility.
Late or corrected data	New version wins by `batch_seq`	Same approach, plus optional tombstones for retractions and bi-temporal columns (`valid_from`, `recorded_at`). That answers "what did we believe on date D?".
Vector search	numpy / DuckDB brute force (exact)	An ANN index (DuckDB `vss` HNSW, pgvector or a managed vector database) built from `article_embeddings`, with re-indexing triggered by new `text_hash`es. Embedding runs as a separate scalable task (batch GPU workers), keyed by the content hash.
Reproducibility	Pinned requirements, deterministic ids and ordering	Container image per release. `parser_version` and `model_name` on every row.

9. Trade-offs and known limitations

- **Full rebuild of silver and gold**: simplest correct design at this size. At scale it becomes a `MERGE` on changed keys (§8).
- **The midpoint of a range is the point estimate**, and the bounds are kept. For a range with mixed currencies, the row becomes `unparseable` rather than guessing.
- **Last-version-wins** uses batch order. If a source re-sends an old version after a newer one (A → B → A), silver keeps B. Handling that needs a source `updated_at` or a sightings table.
- `company_age` is calendar-year granular: `published_year − founded_year`.
- **Metadata is a snapshot.** A company dropped from a later metadata file keeps its last known attributes.
- **The ambiguous dash-date convention is weak** (§4). Rows are flagged rather than silently trusted.